

# LAB13: Stima dei parametri e confronto modello-dati

## Obiettivi

In questo laboratorio impariamo a percorrere la direzione inversa rispetto alla simulazione: dati osservati, vogliamo stimare i parametri del modello che li ha generati e verificare se il modello e' compatibile con i dati.

L'obiettivo non e' solo implementare algoritmi, ma capire:

1. perche' la log-likelihood e' la quantita' giusta da massimizzare;
2. come si traduce in codice la likelihood per un processo di Poisson, per una SDE, per un processo puntuale;
3. cosa ci dice un QQ-plot e quando ci preoccupare;
4. come si stimano i parametri di un modello agent-based quando la likelihood non esiste in forma chiusa;
5. quando l'ABC ha senso e quali sono i suoi limiti pratici.

Lavoreremo sempre con modelli gia' visti nel corso: questo rende concreta ogni formula.

## Il contesto

In tutti i laboratori precedenti abbiamo fissato i parametri e generato dati simulati. Qui facciamo il contrario:

dati osservati  $\xrightarrow{\text{inferenza}}$  stima dei parametri.

I "dati osservati" di questo laboratorio sono sempre dati sintetici che generiamo noi con parametri noti. Questo ci permette di verificare se le stime sono corrette.

## Parametri di lavoro

Ogni parte indica i propri parametri. Come ordine di grandezza:

- per le stime MLE:  $n = 200$ – $500$  osservazioni sono sufficienti;

- per la simulazione della SDE:  $T = 50$ ,  $\Delta t = 0.01$ ;
- per SMM e ABC:  $M = 20$ – $50$  simulazioni per valutazione della funzione obiettivo.

---

## Parte 1 – Log-likelihood per un processo di Poisson

### 1.1 Generare i dati

Generate  $n = 300$  tempi di interarrivo da un processo di Poisson con tasso vero  $\lambda_0 = 2.5$ :

```
import random, math

lambda_true = 2.5
n = 300
inter_arrivals = [random.expovariate(lambda_true) for _ in range(n)]
```

### 1.2 Forma della log-likelihood

La log-likelihood per  $n$  tempi di interarrivo i.i.d.  $\sim \text{Exp}(\lambda)$  e’:

$$\ell(\lambda) = n \log \lambda - \lambda \sum_{i=1}^n s_i.$$

Implementatela e tracciate il grafico di  $\ell(\lambda)$  per  $\lambda \in [0.5, 6]$ .

### 1.3 Stima in forma chiusa

Lo stimatore MLE e’  $\hat{\lambda} = n / \sum_i s_i$ . Calcolatelo e sovrapponetelo al grafico.

### 1.4 Stima numerica

Massimizzate  $\ell(\lambda)$  numericamente con `scipy.optimize.minimize_scalar` sull’intervallo  $(0.1, 10)$ . Verificate che le due stime concordino.

## Domande

1. Il massimo della log-likelihood coincide con la stima in forma chiusa?
2. Cosa succede alla curva se ripetete con  $n = 30$  invece di  $n = 300$ ? E con  $n = 3000$ ?
3. La curva e’ simmetrica attorno al massimo? Cosa implica questo per la distribuzione asintotica di  $\hat{\lambda}$ ?

## Parte 2 – MLE per una SDE (Ornstein-Uhlenbeck)

### 2.1 Il modello

Considerate la SDE di Ornstein-Uhlenbeck:

$$dX_t = -\theta(X_t - \mu_{ou}) dt + \sigma dW_t,$$

con parametri veri  $\theta_0 = 1.5$ ,  $\mu_{ou,0} = 2.0$ ,  $\sigma_0 = 0.8$ .

### 2.2 Generare la traiettoria

```
def simulate_ou(theta, mu_ou, sigma, x0, dt, T):
    n_steps = int(T / dt)
    x = x0
    traj = [x]
    for _ in range(n_steps):
        dW = random.gauss(0, math.sqrt(dt))
        x = x - theta * (x - mu_ou) * dt + sigma * dW
        traj.append(x)
    return traj

dt = 0.01
T = 50.0
traj = simulate_ou(1.5, 2.0, 0.8, x0=2.0, dt=dt, T=T)
```

### 2.3 Log-likelihood

Per ogni incremento osservato, l'approssimazione Euler-Maruyama implica:

$$X_{t+\Delta t} | X_t \sim \mathcal{N}(X_t - \theta(X_t - \mu_{ou})\Delta t, \sigma^2\Delta t).$$

Implementate la log-likelihood sommando i contributi di ogni incremento:

```
def log_normal(x, mean, std):
    return (-0.5*math.log(2*math.pi) - math.log(std)
           - 0.5*((x - mean)/std)**2)

def ll_ou(params, traj, dt):
    theta, mu_ou, sigma = params
    if sigma <= 0 or theta <= 0:
        return -1e10
    ll = 0.0
    for i in range(len(traj)-1):
        mean = traj[i] - theta*(traj[i]-mu_ou)*dt
        std = sigma * math.sqrt(dt)
```

```

    ll += log_normal(traj[i+1], mean, std)
return ll

```

## 2.4 Stima MLE numerica

```

from scipy.optimize import minimize

res = minimize(lambda p: -ll_ou(p, traj, dt),
               x0=[1.0, 1.5, 0.5],
               method='L-BFGS-B',
               bounds=[(0.01, None), (None, None), (0.01, None)])
theta_hat, mu_hat, sigma_hat = res.x

```

## Domande

1. I tre parametri stimati sono vicini ai valori veri?
2. Quale parametro e' stimato con piu' precisione e quale con meno? Perche'?
3. Ripetete la stima su traiettorie piu' corte ( $T = 5$ ) e piu' lunghe ( $T = 200$ ). Come cambia la precisione?
4. Cosa succede se si osserva solo ogni 10° punto (osservazioni sparse)? Usate comunque lo stesso  $\Delta t$  nella likelihood o dovete adattarlo?

## Parte 3 – Diagnostica: QQ-plot e test KS

### 3.1 QQ-plot per dati Weibull

Generate  $n = 400$  osservazioni da una distribuzione Weibull con forma  $k_0 = 2.0$  e scala  $\lambda_0 = 3.0$ :

```

def sample_weibull(k, lam):
    return lam * (-math.log(random.random()))**(1.0/k)

```

```
data = [sample_weibull(2.0, 3.0) for _ in range(400)]
```

Stimate i parametri con MLE (usate la log-likelihood Weibull vista nella dispensa). Poi costruite il QQ-plot:

```

import matplotlib.pyplot as plt

def weibull_quantile(p, k, lam):
    return lam * (-math.log(1-p))**(1.0/k)

def qqplot(data, quantile_func, title="QQ-plot"):
    n = len(data)
    sorted_data = sorted(data)

```

```

theoretical = [quantile_func((i+0.5)/n) for i in range(n)]
plt.figure()
plt.plot(theoretical, sorted_data, '.', ms=3)
lo = min(min(theoretical), min(sorted_data))
hi = max(max(theoretical), max(sorted_data))
plt.plot([lo,hi],[lo,hi],'r--')
plt.xlabel("quantili teorici")
plt.ylabel("quantili empirici")
plt.title(title)
plt.show()

qqplot(data,
        lambda p: weibull_quantile(p, k_hat, lam_hat),
        title=f"QQ-plot Weibull stimata")

```

## Domande

1. I punti cadono sulla bisettrice? Ci sono deviazioni sistematiche nelle code?
2. Ripetete il QQ-plot usando i parametri veri invece di quelli stimati: la figura cambia visibilmente?
3. Tracciate un QQ-plot usando una distribuzione sbagliata (ad esempio normale con media e varianza empiriche). Come appare?
4. Cosa indica una deviazione nella coda destra? E nella coda sinistra?

## 3.2 Residui trasformati per un processo puntuale

Simulate un processo di Hawkes con  $\mu = 0.5$ ,  $\alpha = 0.6$ ,  $\beta = 1.5$ ,  $T = 200$  (usate il codice della dispensa 07):

```

def simulate_hawkes(mu, alpha, beta, T):
    events = []
    t = 0.0
    lam = mu
    while t < T:
        dt = random.expovariate(lam)
        t_cand = t + dt
        if t_cand > T:
            break
        lam_cand = mu + (lam - mu)*math.exp(-beta*dt)
        if random.random() < lam_cand / lam:
            events.append(t_cand)
            lam = lam_cand + alpha
        else:
            lam = lam_cand
        t = t_cand
    return events

```

```
events = simulate_hawkes(0.5, 0.6, 1.5, T=200)
```

Il teorema di rescaling dice che se il modello e' corretto, i tempi trasformati

$$\tau_k = \int_0^{t_k} \lambda^*(s) ds$$

formano un processo di Poisson omogeneo con tasso 1. Quindi i tempi inter-residuo  $\delta_k = \tau_k - \tau_{k-1}$  devono essere  $\sim \text{Exp}(1)$ .

Calcolate  $\tau_k$  integrando numericamente  $\lambda^*(t)$  tra un evento e il successivo, poi fate il QQ-plot dei  $\delta_k$  contro  $\text{Exp}(1)$ :

```
def compute_residuals(events, mu, alpha, beta):
    taus = [0.0]
    for k in range(len(events)):
        t_prev = events[k-1] if k > 0 else 0.0
        t_k = events[k]
        # integra lambda*(t) da t_prev a t_k su griglia fine
        n_steps = 100
        h = (t_k - t_prev) / n_steps
        integral = 0.0
        for j in range(n_steps):
            t_mid = t_prev + (j+0.5)*h
            lam = mu + sum(alpha*math.exp(-beta*(t_mid-tj))
                           for tj in events[:k])
            integral += lam * h
        taus.append(taus[-1] + integral)
    inter = [taus[k]-taus[k-1] for k in range(1, len(taus))]
    return inter

inter_res = compute_residuals(events, 0.5, 0.6, 1.5)
qqplot(inter_res,
        lambda p: -math.log(1-p),
        title="QQ-plot residui vs Exp(1)")
```

## Domande

1. I punti cadono sulla bisettrice?
2. Ripetete con parametri sbagliati ( $\alpha = 0.2$  invece di 0.6). Cosa cambia nel QQ-plot?
3. Cosa indica una deviazione sistematica verso l'alto nella coda destra dei residui? E verso il basso?

## Parte 4 – Momenti simulati per il modello di Deffuant

### 4.1 Il problema

Il modello di Deffuant con  $N$  agenti ha parametri  $\varepsilon$  (soglia di confidenza) e  $\mu_d$  (velocità di convergenza). La likelihood non esiste in forma chiusa. Useremo il metodo dei momenti simulati (SMM).

### 4.2 Generare i “dati osservati”

```
def deffuant_step(ops, eps, mu_d):
    N = len(ops)
    i = random.randint(0, N-1)
    j = random.randint(0, N-1)
    while j == i:
        j = random.randint(0, N-1)
    if abs(ops[i]-ops[j]) <= eps:
        d = ops[j]-ops[i]
        ops[i] += mu_d*d
        ops[j] -= mu_d*d
    return ops

def simulate_deffuant(N, eps, mu_d, T_factor=200):
    ops = [random.random() for _ in range(N)]
    for _ in range(T_factor*N):
        ops = deffuant_step(ops, eps, mu_d)
    return ops

eps_true = 0.25
mu_d_true = 0.40
N = 200

# dati osservati: media su piu' realizzazioni per ridurre il rumore
obs_opinions = []
for _ in range(5):
    obs_opinions += simulate_deffuant(N, eps_true, mu_d_true)
```

### 4.3 Statistiche sommarie

```
def summary_stats(ops, eps_ref=0.05):
    n = len(ops)
    mean = sum(ops)/n
    std = math.sqrt(sum((x-mean)**2 for x in ops)/n)

    sorted_op = sorted(ops)
```

```

n_clust = 1
for i in range(1, len(sorted_op)):
    if sorted_op[i]-sorted_op[i-1] > eps_ref:
        n_clust += 1

q25 = sorted_op[int(0.25*n)]
q75 = sorted_op[int(0.75*n)]
iqr = q75 - q25

return [std, float(n_clust), iqr]

obs_stats = summary_stats(obs_opinions)
print("Statistiche osservate:", [round(s,3) for s in obs_stats])

```

#### 4.4 Funzione obiettivo SMM

```

def smm_obj(params, obs_stats, N, n_sim=20):
    eps, mu_d = params
    if eps <= 0 or eps > 1 or mu_d <= 0 or mu_d > 0.5:
        return 1e10

    sim_stats = []
    for _ in range(n_sim):
        ops = simulate_deffuant(N, eps, mu_d)
        sim_stats.append(summary_stats(ops))

    mean_sim = [sum(s[i] for s in sim_stats)/n_sim
                 for i in range(len(obs_stats))]

    return sum((obs_stats[i]-mean_sim[i])**2
               for i in range(len(obs_stats)))

```

#### 4.5 Ottimizzazione

```

from scipy.optimize import minimize

res = minimize(smm_obj,
               x0=[0.3, 0.3],
               args=(obs_stats, N),
               method='Nelder-Mead',
               options={'xatol':0.02, 'fatol':0.01, 'maxiter':150})

eps_hat, mu_d_hat = res.x
print(f"eps vero={eps_true:.2f} stimato={eps_hat:.3f}")
print(f"mu_d vero={mu_d_true:.2f} stimato={mu_d_hat:.3f}")

```



## 4.6 Mappa della funzione obiettivo

Calcolate e visualizzate la funzione obiettivo su una griglia  $9 \times 9$  di valori  $(\varepsilon, \mu_d)$ :

```
import matplotlib.pyplot as plt
import numpy as np

eps_grid = [0.10 + 0.05*i for i in range(9)]
mu_d_grid = [0.10 + 0.05*i for i in range(9)]

Z = [[smm_obj([e, m], obs_stats, N, n_sim=10)
      for m in mu_d_grid]
     for e in eps_grid]

plt.imshow(Z, origin='lower',
           extent=[min(mu_d_grid), max(mu_d_grid),
                  min(eps_grid), max(eps_grid)],
           aspect='auto')
plt.colorbar(label='obiettivo SMM')
plt.xlabel('mu_d'); plt.ylabel('epsilon')
plt.plot(mu_d_true, eps_true, 'r*', ms=12, label='vero')
plt.plot(mu_d_hat, eps_hat, 'w+', ms=12, label='stimato')
plt.legend(); plt.title('Paesaggio SMM'); plt.show()
```

## Domande

1. Le stime sono vicine ai valori veri?
2. Quale delle tre statistiche sommarie e' piu' sensibile a  $\varepsilon$ ? Quale a  $\mu_d$ ? Come lo verificate?
3. Cosa succede se si usa  $n_{\text{sim}} = 1$  invece di 20? Il paesaggio e' piu' rumoroso?
4. Il minimo della funzione obiettivo e' unico o ci sono piu' valli? Come lo leggete dalla mappa?
5. Aggiungete una quarta statistica sommaria (ad esempio la posizione del cluster piu' grande). Migliora la stima?

---

## Parte 5 – ABC rejection sampler (facoltativa)

### 5.1 Setup

Usate lo stesso modello di Deffuant della Parte 4. Per semplicita' fissate  $\mu_d = 0.4$  e stimate solo  $\varepsilon$  con ABC.

## 5.2 Implementazione

```
def abc_rejection(obs_stats, N, n_accepted=300, eps_abc=0.4):
    accepted = []
    n_trials = 0

    while len(accepted) < n_accepted:
        eps_try = random.uniform(0.05, 0.50)
        ops = simulate_deffuant(N, eps_try, mu_d=0.4)
        sim_st = summary_stats(ops)

        dist = math.sqrt(sum((obs_stats[i]-sim_st[i])**2
                             for i in range(len(obs_stats))))
        if dist <= eps_abc:
            accepted.append(eps_try)
            n_trials += 1

    return accepted, n_accepted/n_trials

posterior, acc_rate = abc_rejection(obs_stats, N)
print(f"Tasso accettazione: {acc_rate:.4f}")
print(f"Media posteriore eps: {sum(posterior)/len(posterior):.3f} (vero: {eps_true})")
```

## 5.3 Effetto di $\epsilon_{ABC}$

Ripetete con  $\epsilon_{ABC} \in \{0.2, 0.4, 0.8\}$  e confrontate le distribuzioni posteriori.

## Domande

1. Con  $\epsilon_{ABC}$  piccolo la posteriore e' piu' concentrata? Come cambia il tasso di accettazione?
2. Con  $\epsilon_{ABC}$  grande la posteriore assomiglia al prior uniforme?
3. Come scegliereste  $\epsilon_{ABC}$  in pratica?
4. Confrontate la stima ABC con quella SMM della Parte 4: quale e' piu' precisa? Quale e' piu' veloce da calcolare?

---

## Cosa dovrete aver capito alla fine

Al termine del laboratorio dovrete aver verificato che:

1. la log-likelihood e' una somma di contributi indipendenti: ogni osservazione entra con  $\log p(y_i | \theta)$ , e per i processi ogni incremento o ogni evento entra con il suo termine specifico;

2. il MLE in forma chiusa esiste solo per modelli semplici; in tutti gli altri casi si massimizza numericamente con L-BFGS-B o metodi analoghi;
3. il QQ-plot e' lo strumento diagnostico piu' immediato: punti sulla bisettrice indicano buon adattamento, deviazioni nelle code indicano che il modello ha la distribuzione sbagliata nelle regioni estreme;
4. per i processi puntuali, i residui trasformati permettono di verificare visivamente se l'intensita' condizionata e' ben specificata;
5. quando la likelihood e' intrattabile, il metodo dei momenti simulati confronta statistiche aggregate tra dati reali e simulati; la scelta di statistiche informative e' cruciale quanto l'algoritmo di ottimizzazione;
6. l'ABC costruisce una distribuzione a posteriori campionando dal prior e accettando solo i parametri che producono simulazioni simili ai dati; e' concettualmente semplice ma computazionalmente costoso.